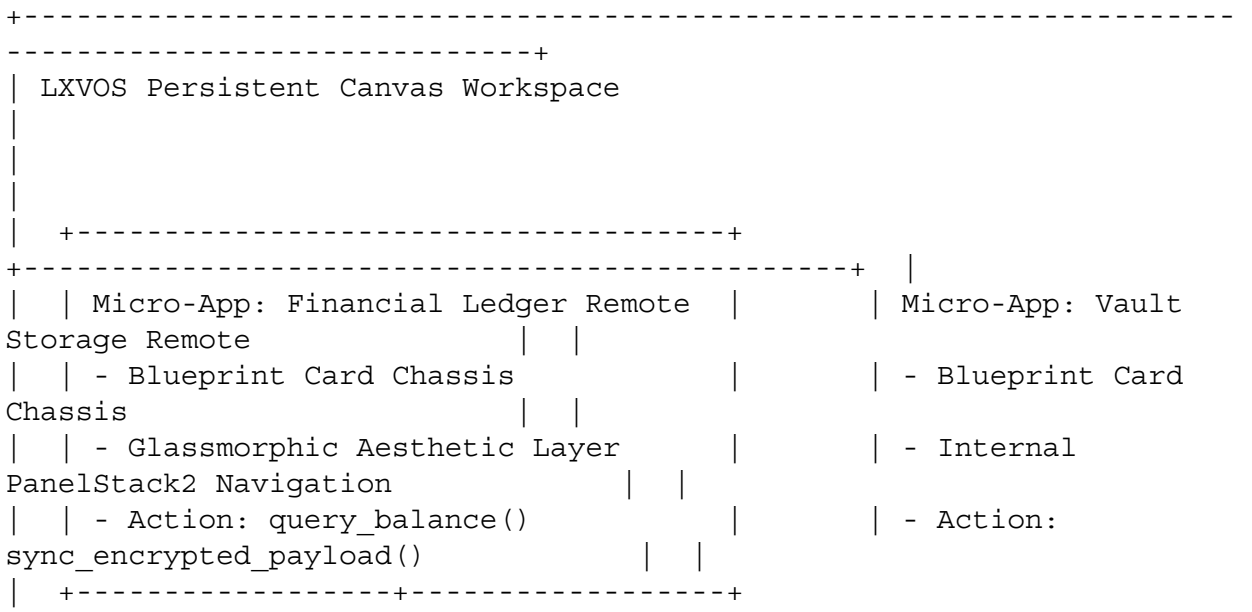
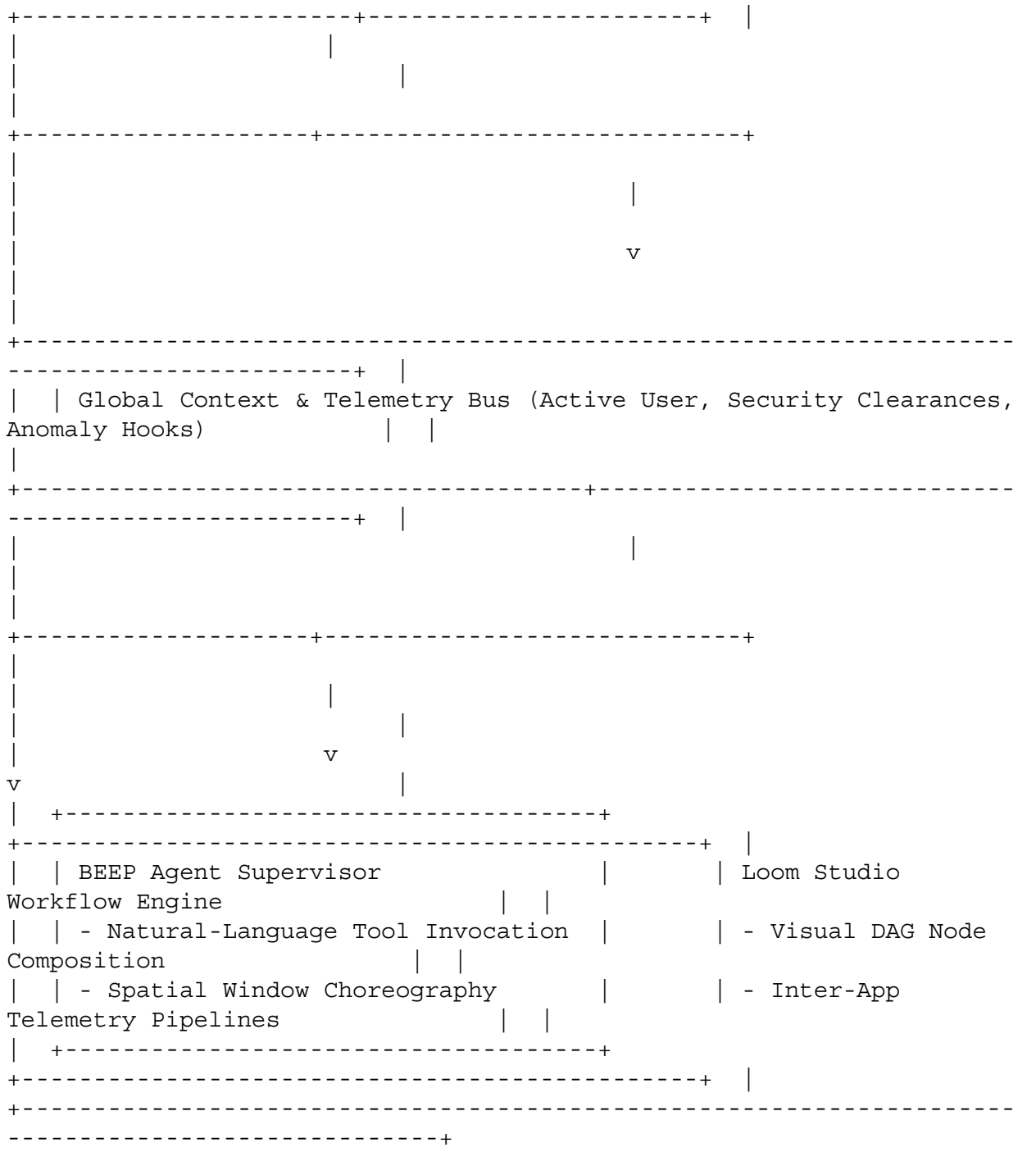
Architectural Blueprint for LXVOS Micro-Apps: Native Federation Remotes, Canvas Windowing, and Agentic Orchestration

Conceptual Architecture and Operational Tenets of LXVOS Micro-Apps

Within the operating system hierarchy of LXVOS, the micro-app serves as the fundamental atomic unit of utility. Conventional enterprise web platforms rely heavily on monolithic viewports, full-page route transitions, and nested tab hierarchies that fragment user context and enforce rigid task boundaries. In contrast, LXVOS models functionality around discrete, self-contained, and context-aware execution modules that run concurrently inside a persistent, spatial Canvas workspace. A micro-app is strictly decoupled from traditional page paradigms; it functions as an intelligent, single-purpose instrument engineered to solve one clear business capability without architectural bloat or feature sprawl.

The visual and physical presentation of each micro-app adheres to the Ancient Roman Glass design system. This design framework rejects generic flat styling in favor of physical depth, refractive glassmorphism, translucent layer hierarchy, and restrained typography. Micro-apps do not remain static within the viewport; they operate as fully interactive window primitives capable of being dragged, scaled, docked, stacked, minimized, and pinned across an infinite or persistent multi-layered Canvas. Each window maintains its internal execution state and document object hierarchy independently, preventing regressions in one surface from disrupting sibling operations.





LXVOS micro-apps are designed agent-first to maintain continuous interoperability with BEEP, the operating system's central agentic brain. Rather than treating user interfaces as black-box visual structures accessible only to human operators, LXVOS exposes every micro-app's capabilities as introspectable, programmatically invocable actions. BEEP possesses the structural authority to open micro-apps, reposition them spatially on the Canvas, pre-fill form parameters, extract runtime telemetry, and trigger multi-step automations.
Simultaneously, micro-apps operate as modular nodes inside Loom Studio, an orchestration environment where technical and operational users visually assemble distributed tools into

composite, event-driven workflows.

Architectural Attribute	Traditional Monolithic Web Application	Standard Module Federation Remote	LXVOS Native Federation Micro-App
Atomic Unit	Page, Route, or Viewport	Remote Micro-Frontend Sub-tree	Single-purpose Context-Aware Window
Bundling Mechanism	Single-bundler compile-time lock	Webpack 5 runtime container lock	Bundler-agnostic ESM and Browser Import Maps
Registration Flow	Static build-time dependency tree	Static remote script manifests	Dynamic runtime manifest via Registry Service
Display Model	Full-page navigation / Viewport swapping	Nested responsive CSS divisions	Spatial Canvas with freeform floating windows
Agent Integration	Unstructured scraping or isolated REST APIs	Opaque JavaScript entry chunks	Introspectable JSON Schemas for BEEP tool calls
UI Design System	Generic external design libraries	Inconsistent, un-scoped CSS themes	Ancient Roman Glass tokens via Blueprint SDK
Security Envelope	Monolithic session trust	Shared origin runtime execution	Aegis Zero-Trust isolation and anomaly hooks

From a strategic software perspective, this architecture directly resolves the compounding maintenance burdens faced by growing enterprises. Monolithic software platforms typically degrade into unmaintainable codebases where minor functional updates require extensive end-to-end regression testing and high-risk continuous deployments. The micro-app paradigm enforces modular isolation, allowing specialized feature teams to develop, test, and deploy targeted domain tools independently.

Furthermore, this model facilitates flexible monetization and operational adaptation. Enterprise tenants install and execute only the discrete micro-apps required for their specific business workflows, avoiding the cognitive overhead and memory footprints associated with bloated multi-suite enterprise software.

Technical and Architectural Prerequisites for the LXVOS Substrate

Achieving dynamic, zero-reload execution of isolated micro-apps within a shared, high-performance Canvas requires a specialized distributed infrastructure. The LXVOS host shell provides the baseline execution context, persistent state stores, spatial windowing manager, and security supervision layers.

Conversely, micro-apps reside across distributed edge endpoints, content delivery networks, or isolated cloud storage buckets, built and deployed entirely out-of-band from the host shell. Connecting these two domains necessitates four foundational infrastructure pillars: dynamic discovery, standardized dependency negotiation, robust inter-app messaging, and continuous context propagation.

Dynamic micro-app registration relies on the federated LXV Registry Service. Rather than compiling remote network locations directly into the host bundle, the host retrieves an environment manifest during initialization. This manifest provides a dynamic routing directory mapping unique micro-app identifiers to their current remote entry endpoints, structural checksums, version semantics, and declared privilege levels. The registration mechanics are formalized through the registry mapping function:
$$\mathcal{R}_{\text{LXVOS}}: \text{AppKey} \to \left\{ \mathcal{V}_{\text{SemVer}}, \mathcal{U}_{\text{remoteEntry}}, \mathcal{S}_{\text{integrity}}, \mathcal{A}_{\text{AegisClearance}} \right\}$$
Shared runtime dependencies represent a crucial performance and stability threshold. In multi-remote micro-frontend environments, loading duplicate vendor libraries across isolated bundles introduces catastrophic memory pressure and inflates network overhead. More critically, running duplicate instances of context-sensitive frameworks such as React or Blueprint disrupts singleton state trees, leading to context separation errors and unexpected lifecycle failures.
LXVOS mandates strict, semver-aware singleton sharing for foundational packages, guaranteeing that libraries like react, react-dom, @blueprintjs/core, and @blueprintjs/icons are loaded exactly once per workspace session, regardless of the number of concurrent remotes mounted on the Canvas.
Inter-application communication requires total structural decoupling. Micro-apps must never expose direct object references to sibling windows or invoke internal methods across boundaries, as this introduces hidden dependency graphs that negate the benefits of isolated deployments.
Communication within LXVOS is mediated by an event-driven publish-subscribe telemetry bus operated by the host shell and continuously monitored by the Aegis security sentinel. Workspace context—including active tenant identity, authenticated user roles, spatial focus, and ambient security posture—is streamed downward to micro-apps via strongly typed subscription contracts governed by the LXV Blueprint SDK.

Native Federation Remote Architecture and Runtime Orchestration

LXVOS establishes its modular micro-frontend infrastructure upon Native Federation, directly addressing the architectural vulnerabilities and tooling limitations of legacy Module Federation. While classic Module Federation implementations are tightly coupled to Webpack's internal runtime mechanics, Native Federation leverages web platform standards, specifically ECMAScript Modules (ESM) and dynamic browser Import Maps.
This ensures that LXVOS micro-apps remain completely build-tool agnostic, empowering distributed development teams to utilize rapid modern build engines such as esbuild, Vite, or the Angular ApplicationBuilder without risking vendor lock-in.

Remote Build Lifecycle and Metadata Compilation

The micro-app compilation process is managed by @softarc/native-federation alongside bundler-specific adapters such as @softarc/native-federation-esbuild. When a micro-app build executes, the builder analyses the project's dependency manifest, distinguishes private implementation code from declared shared packages, and compiles exposed modules into

standard browser-compliant ES modules.
A central configuration file, federation.config.mjs, declares the identity of the remote, the components or modules exposed to the host Canvas, and the explicit dependency sharing rules.

```
// federation.config.mjs - LXVOS Remote Micro-App Configuration
import { withNativeFederation, shareAll } from
'@softarc/native-federation/config';

export default withNativeFederation({
  name: 'lxv-asset-treasury',
  exposes: {
    './MicroApp': './src/app/MicroAppMount.tsx',
    './Descriptor': './src/app/MicroAppDescriptor.ts',
  },
  shared: {
    react: { singleton: true, strictVersion: true, requiredVersion:
'^18.3.0' },
    'react-dom': { singleton: true, strictVersion: true,
requiredVersion: '^18.3.0' },
    '@blueprintjs/core': { singleton: true, strictVersion: false,
requiredVersion: '^5.0.0' },
    '@blueprintjs/icons': { singleton: true, strictVersion: false,
requiredVersion: '^5.0.0' },
  },
  skip: [
    'rxjs/ajax',
    'rxjs/webSocket',
  ],
  features: {
    denseChunking: true,
    denseExternals: true,
  },
});
```

The output of this compilation process is a standardized metadata descriptor, remoteEntry.json, deployed alongside the hashed ESM chunks. The descriptor exposes the complete interface contract required by the host shell, detailing each exposed module's output filename, chunk integrity hashes, and an exhaustive index of shared libraries, including expected version bounds and singleton constraints. The inclusion of denseExternals packages dependencies into unified module bundles, streamlining version negotiation and eliminating chunk waterfall requests during remote initialization.

```
{
  "name": "lxv-asset-treasury",
  "exposes": [
    {
      "key": "./MicroApp",
      "outFileName": "MicroAppMount.js"
    },
```

```
    {
      "key": "./Descriptor",
      "outFileName": "MicroAppDescriptor.js"
    }
  ],
  "shared": [
    {
      "packageName": "react",
      "version": "18.3.1",
      "requiredVersion": "^18.3.0",
      "singleton": true,
      "strictVersion": true,
      "outFileName": "react-18.3.1.js"
    },
    {
      "packageName": "@blueprintjs/core",
      "version": "5.18.0",
      "requiredVersion": "^5.0.0",
      "singleton": true,
      "strictVersion": false,
      "outFileName": "blueprintjs-core-5.18.0.js"
    }
  ]
}
```

Runtime Orchestration and Dynamic Injection

Within the LXVOS host shell, dynamic resolution is governed by @softarc/native-federation-orchestrator. During workspace bootstrapping, the orchestrator retrieves the dynamic environment manifest, fetches remoteEntry.json files across all registered micro-apps, and resolves version parity using semantic version algorithms. Rather than relying on rigid compile-time linkage, the orchestrator generates a unified browser Import Map directly inside the Document Object Model.

```
// src/core/federation/LXVOrchestrationService.ts
import { initFederation } from
'@softarc/native-federation-orchestrator';
import { localStorageEntry } from
'@softarc/native-federation-orchestrator/options';

export interface LXVRuntimeOrchestrator {
  loadRemoteModule: <T = unknown>(remoteName: string, exposedModule:
string) => Promise<T>;
  registerDynamicRemote: (remoteName: string, entryUrl: string) =>
Promise<void>;
}

export async function initializeLXVOrchestrator(
```

```
  manifestEndpoint: string
): Promise<LXVRuntimeOrchestrator> {
  const manifestResponse = await fetch(manifestEndpoint);
  const federationManifest = await manifestResponse.json();

  const orchestratorResult = await initFederation(federationManifest,
{
    storage: localStorageEntry,
    feature: {
      convertFlatSharedInfo: true,
    },
  });

  return {
    loadRemoteModule: orchestratorResult.loadRemoteModule,
    registerDynamicRemote: async (remoteName: string, entryUrl:
string) => {
      await orchestratorResult.initRemoteEntry(entryUrl);
    },
  };
}
```

The orchestrator's storage configuration uses localStorageEntry, which serializes negotiated dependency mappings across sessions to eliminate redundant network overhead during subsequent page reloads. When micro-apps are introduced dynamically mid-session—whether initiated by user navigation or an automated BEEP directive—the host executes initRemoteEntry(), mutating the active browser Import Map without disrupting existing Canvas windows or triggering workspace refreshes.

Canvas Windowing, Docking, and the Blueprint SDK Interface

The physical layout of the LXVOS desktop is engineered for high-density, analytical data display, drawing heavily on the architectural foundation of Palantir's Blueprint UI toolkit (@blueprintjs/core) combined with the proprietary LXV Blueprint SDK. The workspace models each micro-app as an independent, floating window chassis containing specialized control bars, dynamic layout sensors, and sandboxed mounting boundaries.

Blueprint UI Toolkit Implementation in Desktop Workspaces

Blueprint provides the industrial-grade, data-dense UI foundation required for enterprise operations. The LXVOS Canvas leverages specific Blueprint primitives to deliver window chrome and viewport management:
Blueprint's Card serves as the primary visual chassis, configured with interactive elevations, contextual borders, and dark-theme parameters to establish visual boundaries on the multi-layered Canvas.
Window resizing and boundary monitoring are captured through ResizeSensor, which monitors

child container reflows without triggering expensive global window event listeners.
Internal navigation within a micro-app window is maintained via PanelStack2, which isolates drill-down views and sub-navigation state within the remote without modifying the host URL router or interfering with parallel micro-apps.
Window control bars utilize ButtonGroup, Button, Icon, and HTMLSelect components to provide actions such as pin, minimize, maximize, detach, and close.
Flyouts, floating tooltips, and contextual action menus are bound to Blueprint's Overlay2 and ContextMenu2, allowing popover layers to escape the physical clipping boundaries of the window chassis without detaching from its local focus hierarchy.

The Ancient Roman Glass Design Tokens

LXVOS overlays Blueprint’s baseline components with a comprehensive design system embodying the Ancient Roman Glass aesthetic. The interface integrates dark glassmorphism, refractive blur filters, delicate specular highlights, and gold highlights to evoke an enduring, ceremonial workspace.

```
/* src/theme/lxv-roman-glass.css */
:root {
  --lxv-glass-surface: rgba(14, 20, 32, 0.68);
  --lxv-glass-border: rgba(180, 205, 237, 0.16);
  --lxv-glass-specular: rgba(255, 255, 255, 0.07);
  --lxv-glass-blur-radius: 18px;
  --lxv-gold-patina: #d4af37;
  --lxv-gold-patina-glow: rgba(212, 175, 55, 0.25);
  --lxv-text-primary: #f1f5f9;
  --lxv-text-archaic: #cbd5e1;
}

.lxv-window-chassis.bp5-card {
  background: var(--lxv-glass-surface) !important;
  backdrop-filter: blur(var(--lxv-glass-blur-radius)) saturate(190%);
  -webkit-backdrop-filter: blur(var(--lxv-glass-blur-radius))
saturate(190%);
  border: 1px solid var(--lxv-glass-border) !important;
  box-shadow: 0 16px 40px -12px rgba(0, 0, 0, 0.65), 0 0 1px 1px
var(--lxv-glass-border) !important;
  border-radius: 6px;
  display: flex;
  flex-direction: column;
  padding: 0 !important;
}

.lxv-window-header {
  background: linear-gradient(180deg, var(--lxv-glass-specular) 0%,
transparent 100%);
  border-bottom: 1px solid var(--lxv-glass-border);
  padding: 8px 14px;
  display: flex;
```

```
  align-items: center;
  justify-content: space-between;
}

.lxv-window-header .lxv-window-title {
  font-family: 'Cinzel', 'Trajan Pro', Georgia, serif;
  font-size: 12px;
  letter-spacing: 0.12em;
  color: var(--lxv-gold-patina);
  text-shadow: 0 0 8px var(--lxv-gold-patina-glow);
  text-transform: uppercase;
}
```

The LXV Blueprint SDK Contract

The LXV Blueprint SDK defines the standard interface and execution contract between the host Canvas and every federated Native Federation remote. To execute within LXVOS, a micro-app remote must export a descriptor declaring its capabilities, required security posture, and action schemas, alongside a lifecycle mounting method.

```
// @lxv/blueprint-sdk/contract.ts
export type SecurityClassification = 'PUBLIC' | 'INTERNAL' |
'CONFIDENTIAL' | 'RESTRICTED';

export interface LXVWorkspaceContext {
  tenantId: string;
  userId: string;
  classificationClearance: SecurityClassification;
  activeCanvasId: string;
  telemetryBus: BroadcastChannel;
}

export interface LXVActionParameterSchema {
  type: 'object';
  properties: Record<string, { type: string; description: string;
format?: string }>;
  required?: string[];
}

export interface LXVActionDefinition {
  name: string;
  description: string;
  parameters: LXVActionParameterSchema;
  handler: (parameters: Record<string, unknown>) => Promise<unknown>;
}

export interface LXVAegisSecurityProfile {
  classification: SecurityClassification;
```

```
  permissions: string[];
  egressAllowlist: string[];
}

export interface LXVMicroAppDescriptor {
  appId: string;
  displayName: string;
  version: string;
  icon: string;
  initialGeometry: { width: number; height: number; minWidth?: number;
minHeight?: number };
  aegisProfile: LXVAegisSecurityProfile;
  actions: LXVActionDefinition[];
}

export interface LXVMountParameters {
  hostElement: HTMLElement;
  context: LXVWorkspaceContext;
  onContextUpdate: (callback: (updatedContext: LXVWorkspaceContext) =>
void) => void;
  emitWorkspaceEvent: (eventType: string, payload: unknown) => void;
  raiseSecurityAnomaly: (anomaly: { threatCode: string; detail:
string; level: 'WARN' | 'CRITICAL' }) => void;
}

export type LXVMountSignature = (parameters: LXVMountParameters) =>
Promise<() => void>;
```

Agentic Brain Integration (BEEP) and Workflow Composition (Loom Studio)

Unlike conventional graphical interfaces where automation is retrofitted via external screen-scraping or brittle selector-based scripts, LXVOS treats intelligent agent manipulation as a native operational requirement. Every micro-app functions as an introspectable tool provider for BEEP, the platform’s autonomous reasoning supervisor.

BEEP Orchestration Mechanics

When the Native Federation orchestrator loads a remote, the host shell inspects its exposed ./Descriptor entry point. The descriptor's action array is converted into a structured tool-calling definition compliant with standard language-model function schemas.
When a user submits a natural language instruction—such as requesting an asset allocation update or financial reconciliation—BEEP analyzes the request, maps the operational intent to the corresponding micro-app action, and invokes its handler with schema-validated parameters.

```
// src/app/MicroAppDescriptor.ts - Exposed via Native Federation
Remote
```

```
import { LXVMicroAppDescriptor } from '@lxv/blueprint-sdk/contract';

export const descriptor: LXVMicroAppDescriptor = {
  appId: 'lxv-asset-treasury',
  displayName: 'Asset Treasury Vault',
  version: '2.1.0',
  icon: 'bank-account',
  initialGeometry: { width: 720, height: 540, minWidth: 480,
minHeight: 360 },
  aegisProfile: {
    classification: 'RESTRICTED',
    permissions: ['treasury:read', 'treasury:write'],
    egressAllowlist: ['https://vault-api.lxv.internal'],
  },
  actions: [
    {
      name: 'execute_treasury_reallocation',
      description: 'Reallocates liquid assets between internal vault
reserves.',
      parameters: {
        type: 'object',
        properties: {
          sourceVaultId: { type: 'string', description: 'Origin vault
reserve identifier' },
          targetVaultId: { type: 'string', description: 'Destination
reserve identifier' },
          transferAmount: { type: 'number', description: 'Total
capital units to shift' },
        },
        required: ['sourceVaultId', 'targetVaultId',
'transferAmount'],
      },
      handler: async (args) => {
        const { sourceVaultId, targetVaultId, transferAmount } = args
as {
          sourceVaultId: string;
          targetVaultId: string;
          transferAmount: number;
        };
        // Invokes internal state-store logic inside the isolated
micro-app
        return
window.__LXV_TREASURY_CONTAINER__.processTransfer(sourceVaultId,
targetVaultId, transferAmount);
      },
    },
  ],
};
```

BEEP's capabilities extend beyond functional execution to the physical choreography of the Canvas. BEEP coordinates the window manager, monitoring open viewports and automatically mounting, focusing, or minimizing micro-apps to streamline workflows.
If an operational anomaly is detected in an upstream tool, BEEP repositions relevant diagnostic micro-apps side-by-side, aligning their coordinate matrices and highlighting anomalous data structures through automated telemetry pipes.

Loom Studio Composability

In Loom Studio, micro-apps form the physical nodes of an event-driven workflow engine. Users drag micro-app capabilities onto a visual canvas, linking output schemas from one remote to input action handlers on another.
Because all federated remotes expose their structural parameters via the LXV Blueprint SDK, Loom Studio enforces strict type and permission safety at design time. For instance, a security policy micro-app can stream real-time threat scores directly into an infrastructure routing micro-app, enabling automated network reconfigurations without manual API orchestration.

Aegis Security Architecture, Anomaly Telemetry, and Zero-Trust Execution

Deploying federated remote micro-frontends directly inside a shared browser execution environment introduces significant security risks if unconstrained. Traditional iframe boundaries offer process isolation but sever layout cohesion, break Blueprint popover portals, and introduce severe memory overhead.
Native Federation solves performance challenges by loading remotes directly into the host realm as native ECMAScript modules, but this model requires strict security controls to prevent unauthorized access and data exfiltration. The LXVOS Aegis subsystem implements a comprehensive zero-trust runtime framework to secure this shared execution space.
Aegis begins by enforcing W3C Trusted Types policies across the Document Object Model. Dynamic script elements and Import Map injections produced by the Native Federation orchestrator must pass cryptographic and structural verification before execution, preventing DOM-based cross-site scripting (XSS) vectors.
To prevent cross-app CSS collision and unauthorized DOM inspection, the host shell mounts each micro-app inside an isolated Shadow DOM container attached to the Blueprint Card chassis. This encapsulates internal component trees, preventing malicious scripts from querying elements across window boundaries via the host document reference.
Network egress is mediated by an isolated fetch proxy configured on the micro-app's mounting boundary. The LXV Blueprint SDK intercepts external HTTP and WebSocket requests, evaluating destination endpoints against the remote's static egressAllowlist declared in its Aegis descriptor. If a remote attempts to transmit data to an undeclared origin, the proxy blocks the request and triggers an anomaly event.
Aegis mandates that all micro-apps register with real-time anomaly detection hooks. If an unauthorized memory modification, storage violation, or unhandled cryptographic exception occurs within a remote, the micro-app invokes raiseSecurityAnomaly().
Aegis handles this by instantly freezing the offending window chassis, severing its telemetry channels, unmounting its DOM hierarchy, and displaying an urgent, human-readable alert

across the central Canvas notification hub, protecting surrounding operational state.

Reference Implementation: Canvas Host and Federated Remote

The following production implementation demonstrates the integration of the LXVOS host shell, Native Federation runtime, and the Blueprint UI windowing system.

Host Canvas Window Manager

The host window manager mounts on the persistent Canvas layer, dynamically retrieving remote entries via the Native Federation orchestrator, applying Blueprint window chrome, and managing the multi-window coordinate space.

```
// src/components/canvas/CanvasWindowManager.tsx
import React, { useEffect, useRef, useState } from 'react';
import { Card, Elevation, Button, ButtonGroup, Icon, ResizeSensor }
from '@blueprintjs/core';
import { LXVRuntimeOrchestrator } from
'../../core/federation/LXVOrchestrationService';
import { LXVWorkspaceContext, LXVMountSignature, LXVMountParameters }
from '@lxv/blueprint-sdk/contract';
import '@blueprintjs/core/lib/css/blueprint.css';
import '@blueprintjs/icons/lib/css/blueprint-icons.css';
import '../../theme/lxv-roman-glass.css';

interface WindowState {
  appId: string;
  title: string;
  x: number;
  y: number;
  width: number;
  height: number;
  zIndex: number;
  isPinned: boolean;
  unmountCallback?: () => void;
}

interface WindowManagerProperties {
  orchestrator: LXVRuntimeOrchestrator;
  context: LXVWorkspaceContext;
  registeredAppIds: string[];
}

export const CanvasWindowManager: React.FC<WindowManagerProperties> =
({
  orchestrator,
  context,
```

```
  registeredAppIds,
}) => {
  const [windows, setWindows] = useState<Map<string, WindowState>>(new
Map());
  const canvasReference = useRef<HTMLDivElement>(null);

  useEffect(() => {
    registeredAppIds.forEach(async (appId) => {
      if (windows.has(appId)) return;

      try {
        const descriptorModule = await orchestrator.loadRemoteModule<{
descriptor: any }>(
          appId,
          './Descriptor'
        );
        const { displayName, initialGeometry } =
descriptorModule.descriptor;

        const newWindow: WindowState = {
          appId,
          title: displayName,
          x: 100 + windows.size * 36,
          y: 100 + windows.size * 36,
          width: initialGeometry.width || 680,
          height: initialGeometry.height || 480,
          zIndex: 10 + windows.size,
          isPinned: false,
        };

        setWindows((previous) => new Map(previous).set(appId,
newWindow));
      } catch (error) {
        console.error(`[LXVOS] Error loading remote descriptor for
${appId}:`, error);
      }
    });
  }, [registeredAppIds]);

  const elevateZIndex = (appId: string) => {
    setWindows((previous) => {
      const updated = new Map(previous);
      const target = updated.get(appId);
      if (!target) return previous;

      const highestCurrentZ =
Math.max(...Array.from(updated.values()).map((w) => w.zIndex), 10);
      target.zIndex = highestCurrentZ + 1;
```

```
      return updated;
    });
  };

  const terminateWindow = (appId: string) => {
    setWindows((previous) => {
      const updated = new Map(previous);
      const target = updated.get(appId);
      if (target?.unmountCallback) {
        target.unmountCallback();
      }
      updated.delete(appId);
      return updated;
    });
  };

  return (
    <div className="lxv-canvas-workspace-viewport"
ref={canvasReference}>
      {Array.from(windows.values()).map((win) => (
        <CanvasWindowInstance
          key={win.appId}
          windowState={win}
          orchestrator={orchestrator}
          context={context}
          onElevate={() => elevateZIndex(win.appId)}
          onClose={() => terminateWindow(win.appId)}
        />
      ))}
    </div>
  );
};

interface WindowInstanceProperties {
  windowState: WindowState;
  orchestrator: LXVRuntimeOrchestrator;
  context: LXVWorkspaceContext;
  onElevate: () => void;
  onClose: () => void;
}

const CanvasWindowInstance: React.FC<WindowInstanceProperties> = ({
  windowState,
  orchestrator,
  context,
  onElevate,
  onClose,
}) => {
```

```
  const mountNodeReference = useRef<HTMLDivElement>(null);
  const [dimensions, setDimensions] = useState({
    width: windowState.width,
    height: windowState.height,
  });

  useEffect(() => {
    let cleanup: (() => void) | undefined;

    (async () => {
      if (!mountNodeReference.current) return;

      try {
        const remoteAppModule = await orchestrator.loadRemoteModule<{
mount: LXVMountSignature }>(
          windowState.appId,
          './MicroApp'
        );

        cleanup = await remoteAppModule.mount({
          hostElement: mountNodeReference.current,
          context,
          onContextUpdate: (handler) => {
            // Subscribe to workspace context shifts
          },
          emitWorkspaceEvent: (eventType, payload) => {
            context.telemetryBus.postMessage({ eventType, payload,
origin: windowState.appId });
          },
          raiseSecurityAnomaly: (anomaly) => {
            console.warn(`[AEGIS TRAP: ${anomaly.threatCode}]`,
anomaly.detail);
            onClose();
          },
        });

        windowState.unmountCallback = cleanup;
      } catch (mountError) {
        console.error(`[LXVOS] Execution failure within remote
${windowState.appId}:`, mountError);
      }
    })();

    return () => {
      if (cleanup) cleanup();
    };
  }, [windowState.appId]);
```

```
  return (
    <div
      className="lxv-floating-window-frame"
      style={{
        transform: `translate3d(${windowState.x}px,
${windowState.y}px, 0)`,
        width: dimensions.width,
        height: dimensions.height,
        zIndex: windowState.zIndex,
      }}
      onMouseDown={onElevate}
    >
      <ResizeSensor
        onResize={(entries) => {
          const entry = entries[0];
          if (entry) {
            setDimensions({
              width: entry.contentRect.width,
              height: entry.contentRect.height,
            });
          }
        }}
      >
        <Card elevation={Elevation.FOUR} className="lxv-window-chassis
bp5-dark">
          <div className="lxv-window-header">
            <span
className="lxv-window-title">{windowState.title}</span>
            <ButtonGroup minimal={true}>
              <Button icon="pin" small={true}
active={windowState.isPinned} />
              <Button icon="minus" small={true} />
              <Button icon="cross" small={true} intent="danger"
onClick={onClose} />
            </ButtonGroup>
          </div>
          <div className="lxv-window-mount-viewport"
ref={mountNodeReference} />
        </Card>
      </ResizeSensor>
    </div>
  );
};
```

Remote Micro-App Implementation

The remote micro-app implements the mounting signature required by the LXV Blueprint SDK,

using standard Blueprint UI controls to construct an interactive operational view while binding its internal actions to the host's event bus.

```
// src/app/MicroAppMount.tsx - Exposed via Native Federation Remote
import React from 'react';
import ReactDOM from 'react-dom/client';
import { Button, Intent, FormGroup, InputGroup } from
'@blueprintjs/core';
import { LXVMountParameters } from '@lxv/blueprint-sdk/contract';

interface ComponentProps {
  parameters: LXVMountParameters;
}

const TreasuryVaultView: React.FC<ComponentProps> = ({ parameters })
=> {
  const [sourceVault, setSourceVault] = React.useState('');
  const [targetVault, setTargetVault] = React.useState('');
  const [amount, setAmount] = React.useState('');

  const handleExecution = () => {
    parameters.emitWorkspaceEvent('TREASURY_TRANSFER_INITIATED', {
      sourceVault,
      targetVault,
      amount: parseFloat(amount),
    });
    setSourceVault('');
    setTargetVault('');
    setAmount('');
  };

  return (
    <div className="lxv-remote-chassis-content" style={{ padding:
'16px' }}>
      <FormGroup label="Source Vault Identifier"
labelFor="source-vault-input">
        <InputGroup
          id="source-vault-input"
          placeholder="VAULT-SRC-01"
          value={sourceVault}
          onChange={(e) => setSourceVault(e.target.value)}
          leftIcon="shield"
        />
      </FormGroup>

      <FormGroup label="Target Reserve Destination"
labelFor="target-vault-input">
        <InputGroup
          id="target-vault-input"
```

```
          placeholder="VAULT-DST-09"
          value={targetVault}
          onChange={(e) => setTargetVault(e.target.value)}
          leftIcon="send-to-graph"
        />
      </FormGroup>

      <FormGroup label="Capital Quantity (USD Units)"
labelFor="capital-input">
        <InputGroup
          id="capital-input"
          placeholder="0.00"
          value={amount}
          onChange={(e) => setAmount(e.target.value)}
          leftIcon="dollar"
        />
      </FormGroup>

      <Button
        intent={Intent.PRIMARY}
        icon="layout-hierarchy"
        text="Commit Asset Reallocation"
        onClick={handleExecution}
        fill={true}
      />
    </div>
  );
};

export const mount = async (parameters: LXVMountParameters):
Promise<() => void> => {
  const mountRoot = ReactDOM.createRoot(parameters.hostElement);
  mountRoot.render(<TreasuryVaultView parameters={parameters} />);

  return () => {
    mountRoot.unmount();
  };
};
```

Strategic Architectural Evaluation and Trade-Off Analysis

Designing an enterprise operating system environment via browser-native federation involves fundamental trade-offs between memory overhead, build-time flexibility, runtime performance, and system isolation.

Memory utilization and thread execution dynamics require careful optimization in the LXVOS

spatial Canvas. Standard single-page applications reclaim memory during page navigation as previous component subtrees unmount and are garbage collected.
In LXVOS, however, multiple complex micro-apps run concurrently on the Canvas workspace. If remotes bundle duplicate instances of common libraries or run unconstrained background polling, browser memory consumption increases rapidly, causing frame drops and degrading Canvas responsiveness.
LXVOS solves this through window-level virtualization and selective execution suspension. When a window is minimized or obscured beneath active stacks, the host window manager throttles animation frame loops and suspends non-critical telemetry streams.
Furthermore, by enforcing strict singleton sharing across packages such as React and Blueprint, common runtime libraries are shared in memory across all active windows. Version negotiation governed by @softarc/native-federation-orchestrator ensures that differing patch versions resolve to a single compatible runtime instance, eliminating memory duplication.

Architectural Vector	Monolithic Single-Page Architecture	Webpack Module Federation	LXVOS Native Federation Architecture
Initial Bundle Footprint	Extremely High (All features packaged)	Moderate (Shell + initial chunks)	Minimal (Core shell + metadata catalog)
Tooling Flexibility	Locked to a unified workspace bundler	Bound to Webpack 5 runtime container	Bundler-agnostic (Vite, esbuild, CLI)
Dependency Duplication	Zero (Single compile-time tree)	Prone to runtime duplication bugs	Eliminated via runtime SemVer negotiation
Dynamic Remote Loading	Impossible without full deployment	Requires manual script injection	Zero-reload dynamic registration
Style Boundary Protection	High (Global CSS builds)	Low (Colliding global class rules)	High (Shadow DOM + Blueprint scoping)
Agentic Introspectability	Low (Opaque markup)	Low (Static exposed components)	High (Standardized JSON Action contracts)
Runtime Recovery Model	Total crash on unhandled error	Cascading failure across remotes	Per-window isolation and recovery

System resilience is further reinforced by addressing common failure modes inherent to distributed micro-frontend architectures:
Network partitions or edge availability failures represent an ever-present risk for dynamic remotes. If a third-party host or storage bucket becomes unreachable, standard script imports trigger unhandled runtime rejections. The LXVOS Orchestrator wraps remote loading calls with exponential backoff and provides fallback views, rendering a suspended "Glass Obelisk" state within the window chassis while keeping sister micro-apps operational.
Browser Import Maps are immutable once written to the DOM in native environments, creating challenges for micro-apps loaded dynamically mid-session. The LXVOS orchestrator integrates es-module-shims in dynamic mode, allowing live mutations of the import map without triggering full workspace refreshes.
Styling conflicts can also undermine application interfaces when remotes introduce competing global CSS rules. The host platform enforces strict CSS namespace scoping through Blueprint's .bp5-dark containment classes and style sheets processed at mount time, isolating visual artifacts to the window's interior.

Synthesis and Architectural Trajectory

The LXVOS micro-app architecture unites web-native distribution, desktop-grade windowing, and autonomous agent orchestration into a unified enterprise interface. By replacing proprietary bundler runtimes with Native Federation, LXVOS leverages browser standards—specifically ECMAScript Modules and dynamic Import Maps—to provide toolchain independence and fast, zero-reload remote initialization.
Palantir's Blueprint UI toolkit, augmented by the Ancient Roman Glass aesthetic, delivers the structural components and density required for complex spatial desktop interactions.
Working across this interface, the BEEP agent supervisor and Loom Studio workflow engine transform static, isolated applications into an introspectable, composable operational system.
Protected by the Aegis zero-trust runtime framework, LXVOS establishes a scalable, resilient foundation for the next generation of modular intelligent applications.

Works cited

1. Micro Frontends with Angular and Native Federation, https://blog.angular.dev/micro-frontends-with-angular-and-native-federation-7623cfc5f413 2. @softarc/native-federation - npm, https://www.npmjs.com/package/@softarc/native-federation 3. orchestrator/docs/architecture.md at main - native-federation - GitHub, https://github.com/native-federation/orchestrator/blob/main/docs/architecture.md 4. Angular Micro-frontend Application Using Native Federation, https://www.telerik.com/blogs/guide-creating-angular-based-micro-frontend-application-native-federation 5. [Native federation]: How to enable dependency sharing across, https://github.com/angular-architects/module-federation-plugin/issues/1093 6. Blueprint 5.0 · palantir/blueprint Wiki - GitHub, https://github.com/palantir/blueprint/wiki/Blueprint-5.0 7. @angular-architects/native-federation - NPM, https://www.npmjs.com/package/@angular-architects/native-federation 8. The Orchestrator - How Native Federation v4 Decides What Loads, https://dev.to/sakthicodes22/the-orchestrator-how-native-federation-v4-decides-what-loads-and-what-changed-in-480-36dp 9. Frontend System Design Blueprints & Interview Guides, https://frontendprep.io/guides 10. softarc/native-federation-orchestrator - NPM, https://www.npmjs.com/package/@softarc/native-federation-orchestrator?activeTab=dependencies 11. Micro Frontends with Native Federation - DEV Community, https://dev.to/florianrappl/micro-frontends-with-native-federation-56j4 12. Announcing Native Federation 1.0 - ANGULARarchitects, https://www.angulararchitects.io/en/blog/announcing-native-federation-1-0/ 13. Part II: Composing your own native-federation orchestrator - Medium, https://medium.com/@auke997/part-ii-composing-your-own-native-federation-orchestrator-7fac2d0ea62e 14. Documentation - Blueprint, https://blueprintjs.com/docs/versions/4/ 15. Blueprint - A React-based UI toolkit for the web : r/javascript - Reddit, https://www.reddit.com/r/javascript/comments/bohtdc/blueprint_a_reactbased_ui_toolkit_for_the_web/ 16. Blueprint Components, https://liorocks.github.io/blueprint-components/ 17. ReactJS Blueprint ResizeSensor Component - GeeksforGeeks, https://www.geeksforgeeks.org/reactjs/reactjs-blueprint-resizesensor-component/ 18. resizeSensor.tsx - blueprint - GitHub, https://github.com/palantir/blueprint/blob/develop/packages/core/src/components/resize-sensor/resizeSensor.tsx 19. React.js Blueprint Panel stack (v2) Component - GeeksforGeeks, https://www.geeksforgeeks.org/reactjs/react-js-blueprint-panel-stack-v2-component/ 20. React.js Blueprint Panel stack (v2) Component Panels,

https://www.geeksforgeeks.org/reactjs/react-js-blueprint-panel-stack-v2-component-panels/ 21. Blueprint 6.0 · palantir/blueprint Wiki - GitHub, https://github.com/palantir/blueprint/wiki/Blueprint-6.0 22. 5.x Changelog · palantir/blueprint Wiki - GitHub, https://github.com/palantir/blueprint/wiki/5.x-changelog 23. Open Micro Platform: Building AI-Native Micro Frontends Beyond, https://medium.com/@manojmukherjee777/open-micro-platform-building-ai-native-micro-frontends-beyond-module-federation-b0e8b32bcc84 24. Implementing Multi-Framework in Vanilla JS with Native Federation, https://singhdheerendra.medium.com/implementing-multi-framework-in-vanilla-js-with-native-federation-d7580f571a7d 25. loadRemoteModule with a remote of a remote · Issue #379 - GitHub, https://github.com/angular-architects/module-federation-plugin/issues/379 26. How to Dynamically loading remote components using their, https://stackoverflow.com/questions/79291239/how-to-dynamically-loading-remote-components-using-their-selectors-through-nativ